

Input File Format

The test input file is a CSV file and follows the format described in Section 6.4 of the lab description. The server IDs are [S1, S2, S3, S4, S5, S6, S7] and Client IDs are [A, B, C, D, E, F, G, H, I, J]. These will remain consistent across all test cases.

Example Test Case

Set Number	Transactions	Live Servers	Byzantine Servers
1	(A, I, 2)	[S1, S2, S3, S4, S5, S6, S7]	[S7]
	(B, E, 6)		
	(H, D, 3)		
	(I, G, 2)		

Table 1: Example Test Case

Client Transaction Processing

- **Initial State:** Each client starts with a balance of 10 units.
- **Transaction Handling:** Transactions are processed sequentially at each client.

Clients must pick up and execute these transactions in the order they appear in the input file. Transactions across clients should be concurrent.

Server Transaction Processing

- **Initial State:** Each server maintains all the client balances in the datastore.
- **Transaction Handling:** The live servers use the linear-PBFT protocol, as outlined in the lab description, to agree on a total order for executing transactions within each set. Each server processes the transactions, updating its datastore accordingly. Upon execution, the server responds with a "pass" or "fail" to the client based on whether the transaction was successfully executed (i.e., if sufficient balance was available).

Refer to **Section 6.4** in the lab description for detailed instructions on running the test cases from the input file.

Note: Ensure your clients do not process the next sets of transactions unless explicitly instructed by the user. Additionally, your implementation must expose the functions specified in Section 2.2 of the lab description. These functions will be utilized within this timeframe to validate the state of your servers.

We are particularly interested in the output of the functions described in Section 2.2 of the lab description, particularly the `PrintDB()` and `PrintStatus()` functions.

The expected output of the `PrintDB()` function is shown below:

Server	A	B	C	D	E	F	G	H	I	J
S1	8	4	10	13	16	10	12	7	10	10
S2	8	4	10	13	16	10	12	7	10	10
S3	8	4	10	13	16	10	12	7	10	10
S4	8	4	10	13	16	10	12	7	10	10
S5	8	4	10	13	16	10	12	7	10	10
S6	8	4	10	13	16	10	12	7	10	10
S7	10	10	10	10	10	10	10	10	10	10

Table 2: Expected state of balances on each server after executing set 1

The expected output of the `PrintStatus()` function when provided with the input sequence number 3 (`PrintStatus(3)`) is shown below:

Server	S1	S2	S3	S4	S5	S6	S7
Status	E	E	E	E	E	E	PP

Table 3: Expected Output of `PrintStatus(3)`

Test Case Descriptions

- Test 1: Basic Agreement
- Test 2: Agreement with Byzantine Servers
- Test 3: Agreement with Disconnected Servers
- Test 4: Agreement with Byzantine and Disconnected Servers
- Test 5: Too many concurrent requests
- Test 6: No Agreement with too many Byzantine servers
- Test 7: No Agreement if no super-majority
- Test 8: Agreement with a faulty leader - Part 1
- Test 9: Agreement with a faulty leader - Part 2
- Test 10: Performance Test

The descriptions provided above outline the test cases from the CSV input file. These test cases aim to cover all major scenarios and should be sufficient for validating your implementation.